PQMigrate: Evidence-Driven and Abstention-Aware Planning for Post-Quantum Cryptographic Migration

Dr. Deepak Uplaonkar, Ratnesh Patnaik, Saravanan Kumar, Vaibhavi Shah, and Prince Ostwal

Abstract. PQMigrate is a bounded source-analysis and advisory-planning prototype. Its completed quantitative study is a single-author synthetic pilot of 72 Python RSA snippets, not an independent real-code evaluation. The historical results describe an authored mixture of supported operations, unsupported operations, negative controls and malformed inputs. They establish neither generalization nor migration safety. This revision separates source-level operations from application roles, relates the implementation to cryptographic inventory and API-misuse analysis, and specifies an independent evaluation protocol. Real-code annotation and the external Semgrep comparison remain ongoing and unscored. Historical performance results are omitted; the evaluation status and protocol are reported instead. Planner recommendation correctness, Go accuracy, deployed interoperability and PQC transformations have not been evaluated by this pilot.

Index terms: post-quantum cryptography, cryptographic inventory, static analysis, role inference, selective prediction, reproducibility.

I. Introduction

NIST FIPS 203 defines ML-KEM for key encapsulation; FIPS 204 defines ML-DSA for signatures [1], [2]. FIPS 205 defines the hash-based signature standard SLH-DSA [5]. These functions are distinct. Replacing an RSA signature interface and redesigning an RSA key-transport interface require different evidence about key formats, peers, message sizes and failure handling. A KEM establishes a shared secret and does not offer arbitrary-message encryption as a drop-in replacement [1]. Finding an algorithm name is consequently an inventory lead, not a complete migration decision.

Cryptographic-agility work treats dependency management and coordinated change as research problems [3], [4]. NIST's NCCoE migration project separates cryptographic discovery from interoperability work [8]. PQMigrate addresses part of the evidence-management problem: observations link to source locations, bounded inference attempts to resolve their use, and an advisory planner records conditions and blockers. It does not establish that suggested replacements will interoperate or be secure in the target application.

Standards and deployment status must be dated. At the verification date, 21 October 2026, NIST lists Falcon and HQC standardization as ongoing [6]. RFC 9964 specifies ML-DSA identifiers for JOSE and COSE [7]. This protocol specification is not evidence that a particular dependency, peer or deployment supports those identifiers. No JOSE/PQC integration or transformation is evaluated here.

II. Contribution and evidence boundary

The contribution is an inspectable prototype, an evidence-linked representation of observations and conditional advice, and a reproducible evaluation design. Python and Go parsing support is an implementation capability; the completed numerical pilot evaluates only synthetic Python RSA snippets. A source-level invocation is not proof that a production execution reached it. Test files may contain real crypto invocations, while an import, comment or algorithm descriptor alone is not an operation.

The paper asks whether bounded evidence can distinguish inventory leads, operations and application roles, and how unresolved evidence should be reported. It does not answer whether the prototype is accurate across repositories or whether its advice reduces migration risk. The present manuscript reports the system design, evaluation methodology and limitations. Historical performance tables and numerical findings have been removed. No replacement accuracy estimates are available before independent gold labels and operation-unit definitions are locked.

III. Related work

Discovery and migration dependencies. Cryptoscope describes compiler-based analysis using control and data flow to identify cryptographic usage [9]. Hasan et al. model security dependencies and present migration case studies [4]. These works motivate connecting inventory entries to use and dependencies. PQMigrate's bounded heuristics are not equivalent to whole-program compiler analysis, and this pilot provides no shared-corpus comparison or performance ranking against these systems.

Cryptographic API correctness. CogniCrypt combines assistance for generating cryptographic code with checking its integration [10]. CrySL specifies correct API usage and supports context- and flow-sensitive checking [11]. CryptoGuard uses program-slicing refinements for detecting cryptographic vulnerabilities in Java projects [12]. CryptoAPI-Bench supplies Java API-misuse cases [13]. These tasks concern usage constraints or weaknesses. PQMigrate instead attempts to identify migration-relevant operation roles and produce conditional advice. This distinction does not make it a misuse detector, and none of the cited Java results transfers to the Python/Go setting without measurement.

Inventory tools and interchange. CodeQL documents a Python query for broken or weak cryptographic algorithms [14]. The Sonar cryptography plugin and CBOMkit provide cryptographic discovery and CBOM tooling [15], [16]. CycloneDX CBOM represents cryptographic assets and relationships [17]; SARIF standardizes static-analysis result interchange [18]. An export can satisfy a schema while containing an incorrect source interpretation. Structural validation is therefore separate from detection accuracy, role correctness and downstream ingestion. These tools are relevant comparators or integration targets, not baselines already evaluated in this manuscript.

Reject options and reporting. Chow studies the error/reject trade-off [20]; selective-classification research formalizes coverage and error among accepted predictions [19], [21]. This motivates reporting answered-case correctness together with coverage. PQMigrate does not satisfy the assumptions of a learned selective classifier or inherit a statistical guarantee from those papers. In particular, absence of a resolved role differs from a planner's decision to abstain from advice. The completed pilot measures the former, not the latter.

External rule-engine baseline. Semgrep runs published source patterns through its analysis engine [26]. The separate baseline implementation pins engine version, rules, source hashes and execution limits. Custom rules define the evaluated detector; results cannot be generalized to all Semgrep rule sets or commercial capabilities. Its execution ledger is not an accuracy evaluation. Native spans must be aligned to locked operation units before comparison.

IV. System model and inference

Discovery produces source-linked leads, inference attempts identity and role resolution, and the intermediate representation links findings to a versioned migration knowledge base. The planner exposes advice with conditions, evidence and blockers. JSON, SARIF and CBOM exports serialize observations; they are not additional evidence about the target's behavior [17], [18], [24]. Scanned source is treated as input rather than imported or executed.

For the inspected Python resolver, assignment collection tracks selected RSA key-generation calls and selected random-byte sources. A subsequent pass associates calls with tracked names in a bounded scope. This is name-based evidence, not a general points-to analysis. Imports, assignments, wrappers, mutation and branch conditions may invalidate the inferred association. A role assigned from such evidence remains a fallible inference [24].

An encryption of bytes from a tracked random source is a heuristic proxy used by the historical implementation for key transport. Randomness does not establish that those bytes are key material. The reviewer SOP instead requires evidence of their application use. This limitation requires application-level evidence before assigning key transport.

V. Methodology and interpretation

Synthetic pilot and current study. PQMigrateBench v0.1 is a single-author synthetic pilot of 72 Python RSA snippets, covering operations, negative controls and malformed inputs. The same developer authored and labeled the examples. These deliberately constructed families are not a random sample of real software and cannot establish generalization [24]. Historical performance scores, result tables, plots and numerical findings are excluded from this manuscript.

The inspected implementation revision is bcfb5e40fbc8eb74e1c9 19114f54031d03ab44e9 [24]. Source and corpus identity are recorded in the reproducibility audit. Source inspection supports implementation statements, but externally held historical gate logs were not independently verified. No historical gate-run outcome is claimed here.

Units and coupled outputs. The pilot's scoring unit is a snippet, whereas the real-code protocol uses source-level invocation units. A key may be generated, loaded, used to sign and serialized in one function: those invocations must not silently become one scored call. Historic coupled presence/known-role outputs must be distinguished from operation detection independent of role. An unresolved receiver is not a proven negative. A valid but unsupported operation remains eligible for annotation and failure accounting.

JWT and aggregation limits. Source inspection shows a literal RS-prefixed algorithm check for recognized JWT encoding. RFC 7518 defines PS256 as an RSA-PSS algorithm [22], so an RS-only check does not cover all RSA JWT signatures. This is an implementation-coverage limitation, not a new measured performance estimate. Mixed roles, wrappers, cross-scope flow, aliasing, reassignment and dynamic behavior can defeat bounded inference. The implementation's ordering and aggregation choices are not a soundness proof [24].

Output masks. The session-secret and protocol variants in the pilot mask information in existing outputs. They demonstrate consequences for the reported fields; they are not independent implementations that disable analysis features. Such masks must not be presented as independently implemented feature-removal ablations. A causal claim about a separately implemented analyzer would require a new experiment.

Mutation policy. The inspected policy checks role, construction support, interoperability evidence, tests and operator authorization, refusing mutation when requirements are false or unknown [24]. That control-flow property does not establish correct recommendations or secure transformed code. Tests, schema validity, source preservation, peer compatibility and cryptographic correctness are different obligations. Planner advice and unsafe-recommendation rates require separate independently adjudicated outcomes; this pilot supplies none.

Statistical scope. Correctness among selected answered cases does not establish reliability for a target population. Reject-option literature motivates joint coverage/error reporting but supplies no bound for this authored sample [19]-[21]. Zero observed events would not establish zero underlying risk [23]. No real-code precision, recall, TPR, agreement or confidence interval is asserted here.

The study's next stage is independent source annotation. The following sections distinguish implementation and execution status from accuracy evidence, then specify limitations and the ongoing review protocol.

VI. Evaluation status and reproducibility

Independent real-code evaluation is ongoing. The candidate-selected packet and the separate ordinary-source/independent-search packet serve different purposes: reviewing discovered candidates can assess candidate validity, whereas investigating missed operations requires an independent search design. The assigned production units, reviewer submissions and adjudicated gold must be frozen and traceable before scoring.

The separate Semgrep baseline implementation pins Community Edition 1.181.0, custom Python/Go rules, full repository commits, whole-file hashes, dependency versions and execution limits. It includes source-only manifests, engineering tests, a per-input ledger, and lossless raw stdout/stderr archives. Setup, key-loading and streaming stages must follow the SOP unit definitions rather than being silently merged with calls that consume their outputs. Its exact rule set defines the baseline [26].

Static baseline runs have been preserved for the synthetic pilot and pinned candidate-containing source files. These are unscored execution artifacts. Successful execution, raw finding counts and passing engineering tests do not establish precision, recall, TPR, operation-role accuracy or agreement. No result is substituted for the removed historical tables.

Every input remains accounted for, including unavailable sources, malformed inputs, partial parsing, timeouts and crashes. The runner preserves findings from incomplete scans without converting those scans into clean negatives. Duplicate and nested findings retain their native spans for a later, frozen alignment procedure. Target source is neither imported nor executed. Hash verification checks the source bytes and analysis artifacts, not the correctness of an inferred application role.

The revised manuscript and its source are part of the Paper PR; the baseline rules, runner and raw archives are supplied in the separate Baseline PR. Reproduction requires the corresponding branch and pinned dependencies. Independent labels remain outside these artifacts, and no A/B submissions are used for rule tuning or manuscript accuracy claims.

VII. Discussion and limitations

The synthetic pilot diagnoses what the bounded resolver can and cannot express. It cannot support claims of superiority over external tools, real-code accuracy, reduced migration effort or safer recommendations. Better detection F1 can coexist with worse role assignment; conversely, selective non-answers can inflate correctness among answered cases while leaving useful operations unresolved. Any future performance report must state the task and denominator explicitly.

Internal and construct validity. A single author selected examples, defined labels and implemented the analyzer. Repeated scenario templates and a deliberately supported/unsupported mixture can favor the implementation. Snippet presence was historically coupled to a known role for some systems. Call-level operation presence, application role and planner advice are distinct constructs. Masking outputs is not a feature-removal experiment. Neither non-execution nor passing syntax/schema checks establishes semantic correctness. The random-byte key-transport proxy is specifically insufficient under the reviewer SOP.

External validity. The completed numerical evidence is Python/RSA-specific and synthetic. It does not establish Go performance, wrapper resolution, package-boundary behavior, protocol coverage or operational deployment characteristics. The 500 candidate-selected anchors focus on scanner-discoverable locations; even complete annotation of them cannot reveal all missed operations. The separate ordinary-source/independent-search packet is needed to study omissions. Whole-file Semgrep scans provide more context than isolated snippets but do not solve sampling bias or establish whole-repository coverage.

Statistical validity. Correlated examples, author selection and few repository families limit inference. No significance test, real-code confidence interval or inter-annotator agreement result is available. Once units and gold are locked, report raw denominators, label prevalence, undefined metrics and repository dependence alongside any interval. A planned repository bootstrap is not automatically reliable with few independent clusters; resampling must preserve paired systems and repository families. Favorable and unfavorable outcomes must be reported under the same frozen evaluation protocol.

Reproducibility and standards drift. Pin the code, corpus, rule files, mapping, dependencies and standards access date. Mutable product documentation is evidence of documented capabilities at access time, not of a frozen tool's measured behavior. The inspected source and corpus are available at [24]; externally stored historical gate logs remain a verification gap. The IEEE-formatted bibliography identifies primary sources and versions; the accompanying audit distinguishes standards, author manuscripts, documentation, repository evidence and publisher metadata.

Planning and security validity. A resolved role is necessary evidence for some migration advice, not sufficient evidence of a correct redesign. KEM selection does not itself settle key confirmation, authentication, serialization, failure handling or protocol integration [1], [4]. ML-DSA identifiers in a protocol specification do not prove deployment readiness [7]. No cryptographic security proof, verified PQC transformation, production interoperability measurement or user study is contributed here. Separate proposed cryptography investigations contribute no result to this paper.

VIII. Ongoing real-code evaluation protocol

The 19-23 October 2026 annotation window is an organizational schedule, not evidence of completion. Reviewer eligibility requires not having built the scanner/resolver or seen predictions for the target repositories. Both reviewers independently receive the same 500 anchors, not disjoint halves. No A/B submissions were inspected for this revision or the baseline work. Reviewer interaction about individual cases remains prohibited until both packets are submitted and locked.

A separate unscored calibration batch of approximately 25 anchors must precede production work. It is not the production packet's first 25 rows. The organizer freezes an agreed feasible list of production candidate IDs before review; unassigned rows remain unreviewed and cannot be selected afterward based on favorable labels. Each reviewer works on a private copy with identifying columns and row order preserved.

At each pinned repository commit, reviewers inspect the whole containing function and follow imports, declarations, assignments, wrappers and callers as needed. The hash refers to the entire source file. Source is read and searched; target builds, installers and code are not executed. A proven source-level crypto call or wrapper is an operation even in tests; an import or familiar method name alone is not.

The SOP labels presence as yes/no/uncertain. Operation values are sign, verify, encrypt, decrypt, key_establish, hash, mac, kdf, key_generate, other_crypto, none and uncertain. Role values are signature, key_transport, key_establishment, encryption, hash, mac, kdf, key_management, other, none and uncertain. Unknown receivers or opaque boundaries require an uncertainty reason, rather than a guessed crypto identity. A non-crypto call uses no/none/none. Key creation maps to key_generate/key_management. Application evidence is required for key transport; encrypting random bytes alone is insufficient.

For multiple distinct operations at an anchor, presence is yes, operation and role are uncertain, and a MULTIPLE note identifies exact spans for adjudicator expansion. A wrong-line anchor is needs_context rather than silently relocated. Every completed row needs context and decisive source evidence. Operational definitions for setup calls, streaming stages and one-to-many alignments must be frozen before scored mapping; unresolved questions have been flagged to Ratnesh in repository issue 1 [24]. His instruction is to use the SOP taxonomy.

The study lead receives and locks both original files privately before comparing them. Preserve original disagreements and rationales, then adjudicate; do not replace original labels when computing agreement. Report raw agreement and Cohen's kappa [25] per applicable nominal field with prevalence, completed-pair counts, missing/unreviewed rows and any undefined denominator. No agreement has yet been measured. Calibration, assigned production rows and adjudicated expansions must be traceable as different stages.

After the first wave, the separate 240-anchor independent-search/ordinary-source packet is intended to investigate missed operations. Its design, frozen units, discovery effort and review coverage must be documented before estimating recall or TPR. The candidate-selected packet alone cannot support these claims. Repository families and near duplicates should remain in the same split; final sample sizes must be reported as observed rather than treated as achieved targets.

Baseline and failure accounting. The Semgrep comparison uses the same pinned Python/Go source revisions, published custom rules and the SOP vocabulary [26]. Preserve native findings and spans, raw stdout/stderr, exact versions, input/rule hashes, runtime limits and failure ledgers. Parse failures, timeouts, crashes, skipped files and unavailable sources must remain visible. A partial scan may contain findings and still be incomplete; neither a failure nor a no-match is automatically a gold negative. Duplicate/nested findings require a prespecified alignment policy, not favorable post hoc matching.

Do not tune rules from A/B submissions. Gold must be locked before accuracy calculations, and reporting must include unfavorable results and missingness. The current revision contains no Semgrep accuracy estimate, no real-code precision/recall/TPR, and no inter-annotator agreement. Future performance reporting requires an auditable, comparable experiment and a transparent version note.

IX. Conclusion

PQMigrate provides an inspectable bounded resolver within an advisory-planning prototype. The 72-case Python RSA study is a single-author synthetic pilot; its historical numerical results are omitted from this manuscript. Independent real-code annotation, operation-unit adjudication and an unscored reproducible Semgrep baseline are ongoing. Stronger accuracy, safety, interoperability and transformation claims require separate evidence.

References

[1] National Institute of Standards and Technology, Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final

[2] National Institute of Standards and Technology, Module-Lattice-Based Digital Signature Standard, FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204. [Online]. Available: https://csrc.nist.gov/pubs/fips/204/final

[3] D. Ott, C. Peikert, and other workshop participants, "Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility," Computing Community Consortium workshop report, arXiv:1909.07353, 2019, doi: 10.48550/arXiv.1909.07353. [Online]. Available: https://arxiv.org/abs/1909.07353

[4] K. F. Hasan et al., "A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies," IEEE Access, 2024, doi: 10.1109/ACCESS.2024.3360412; author version arXiv:2307.06520v2. [Online]. Available: https://arxiv.org/abs/2307.06520v2

[5] National Institute of Standards and Technology, Stateless Hash-Based Digital Signature Standard, FIPS 205, Aug. 2024, doi: 10.6028/NIST.FIPS.205. [Online]. Available: https://csrc.nist.gov/pubs/fips/205/final

[6] National Institute of Standards and Technology, "Post-Quantum Cryptography," project status. Accessed: Oct. 21, 2026. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography

[7] M. Prorock and O. Steele, "ML-DSA for JSON Object Signing and Encryption (JOSE) and CBOR Object Signing and Encryption (COSE)," RFC 9964, May 2026, doi: 10.17487/RFC9964. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9964.html

[8] NIST National Cybersecurity Center of Excellence, "Migration to Post-Quantum Cryptography." Accessed: Oct. 21, 2026. [Online]. Available: https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc

[9] M. Moffie et al., "Cryptoscope: Analyzing cryptographic usages in modern software," arXiv:2503.19531v1, Mar. 2025, doi: 10.48550/arXiv.2503.19531. [Online]. Available: https://arxiv.org/abs/2503.19531

[10] S. Krüger et al., "CogniCrypt: Supporting Developers in using Cryptography," author manuscript, 2017. [Online]. Available: https://www.eclipse.org/cognicrypt/resources/publications/knr%2B17cognicrypt.pdf

[11] S. Krüger, J. Späth, K. Ali, E. Bodden, and M. Mezini, "CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs," in ECOOP, LIPIcs, vol. 109, 2018, pp. 10:1-10:27, doi: 10.4230/LIPIcs.ECOOP.2018.10. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2018.10

[12] S. Rahaman et al., "CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects," arXiv:1806.06881v5, 2019, doi: 10.48550/arXiv.1806.06881. [Online]. Available: https://arxiv.org/abs/1806.06881v5

[13] S. Afrose, S. Rahaman, and D. D. Yao, "CryptoAPI-Bench: A Comprehensive Benchmark on Java Cryptographic API Misuses," in Proc. IEEE SecDev, 2019, pp. 49-61; benchmark artifact. [Online]. Available: https://github.com/CryptoAPI-Bench/CryptoAPI-Bench

[14] GitHub, "Use of a broken or weak cryptographic algorithm," CodeQL Python query documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://codeql.github.com/codeql-query-help/python/py-weak-cryptographic-algorithm/

[15] CBOMkit contributors, "Sonar Cryptography Plugin," source and documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://github.com/cbomkit/sonar-cryptography

[16] CBOMkit contributors, "CBOMkit," source and documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://github.com/cbomkit/cbomkit

[17] OWASP Foundation, "Cryptography Bill of Materials (CBOM)," CycloneDX documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://cyclonedx.org/capabilities/cbom/

[18] OASIS, Static Analysis Results Interchange Format (SARIF) Version 2.1.0 Plus Errata 01, OASIS Standard, Aug. 2023. [Online]. Available: https://docs.oasis-open.org/sarif/sarif/v2.1.0/errata01/os/sarif-v2.1.0-errata01-os-complete.html

[19] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," arXiv:1705.08500v2, 2017, doi: 10.48550/arXiv.1705.08500. [Online]. Available: https://arxiv.org/abs/1705.08500v2

[20] C. K. Chow, "On Optimum Recognition Error and Reject Tradeoff," IEEE Trans. Inf. Theory, vol. 16, no. 1, pp. 41-46, Jan. 1970, doi: 10.1109/TIT.1970.1054406. [Online]. Available: https://research.ibm.com/publications/on-optimum-recognition-error-and-reject-tradeoff

[21] R. El-Yaniv and Y. Wiener, "On the Foundations of Noise-free Selective Classification," J. Mach. Learn. Res., vol. 11, no. 53, pp. 1605-1641, 2010. [Online]. Available: https://www.jmlr.org/papers/v11/el-yaniv10a.html

[22] M. Jones, "JSON Web Algorithms (JWA)," RFC 7518, May 2015, doi: 10.17487/RFC7518. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7518.html

[23] J. A. Hanley and A. Lippman-Hand, "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators," JAMA, vol. 249, no. 13, pp. 1743-1745, Apr. 1983, doi: 10.1001/jama.1983.03330370053031. [Online]. Available: https://jamanetwork.com/journals/jama/article-abstract/385438

[24] PQMigrate project, "PQMigrate source code and evaluation artifacts," Git repository, evaluated revision bcfb5e40fbc8eb74e1c919114f54031d03ab44e9, 2026. [Online]. Available: https://github.com/Ratneshp1122/PQMigrate/tree/bcfb5e40fbc8eb74e1c919114f54031d03ab44e9

[25] J. Cohen, "A Coefficient of Agreement for Nominal Scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37-46, Apr. 1960, doi: 10.1177/001316446002000104. [Online]. Available: https://journals.sagepub.com/doi/abs/10.1177/001316446002000104

[26] Semgrep, "Run rules," documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://docs.semgrep.dev/running-rules